

Supervised Learning

Wednesday, 8 April 2026 17:06

1. Preprocessing

In this section, we worked with the Israeli CBS economic literacy dataset. The goal was to predict whether a person has debt based on the available survey features.

The label was created from the Hovot variable. We defined:

$y_{\text{debt}} = 1$ if the respondent has debt, and

$y_{\text{debt}} = 0$ otherwise.

After filtering to labeled observations, we had:

- 1153 labeled samples
- 923 without debt
- 230 with debt
- debt prevalence: $230 / 1153 = 0.1995$
- 444 engineered features

Before modeling, we removed variables that could cause direct leakage. In particular, the original Hovot column was removed from the feature matrix, as well as debt-detail follow-up variables such as Hovot2–Hovot9 and HovotLechatzim.

We also removed variables that were not useful as predictors.

SerialNumber was kept only as an ID variable. The survey expansion weight nn / "מקדם גיפוח" was not used as a regular feature, because it describes the sampling design rather than a personal economic property of the respondent.

- Categorical variables were converted into numeric form using one-hot encoding.
- Ordered variables and binned numeric variables were mapped into meaningful numeric scores where possible.
- Missing values, “don’t know”, “not relevant”, and similar response patterns were kept as indicators when they had possible meaning in the survey structure.
- Other missing values in numeric engineered features were filled with neutral values, usually based on the median among labeled observations.
- Free-text “other/specify” fields were excluded from the model features and audited separately, because they were not directly suitable for the numerical models used in this section.

At the end of preprocessing, the supervised dataset was cleaned of leakage values, free-text and had no missing values in the model feature matrix.

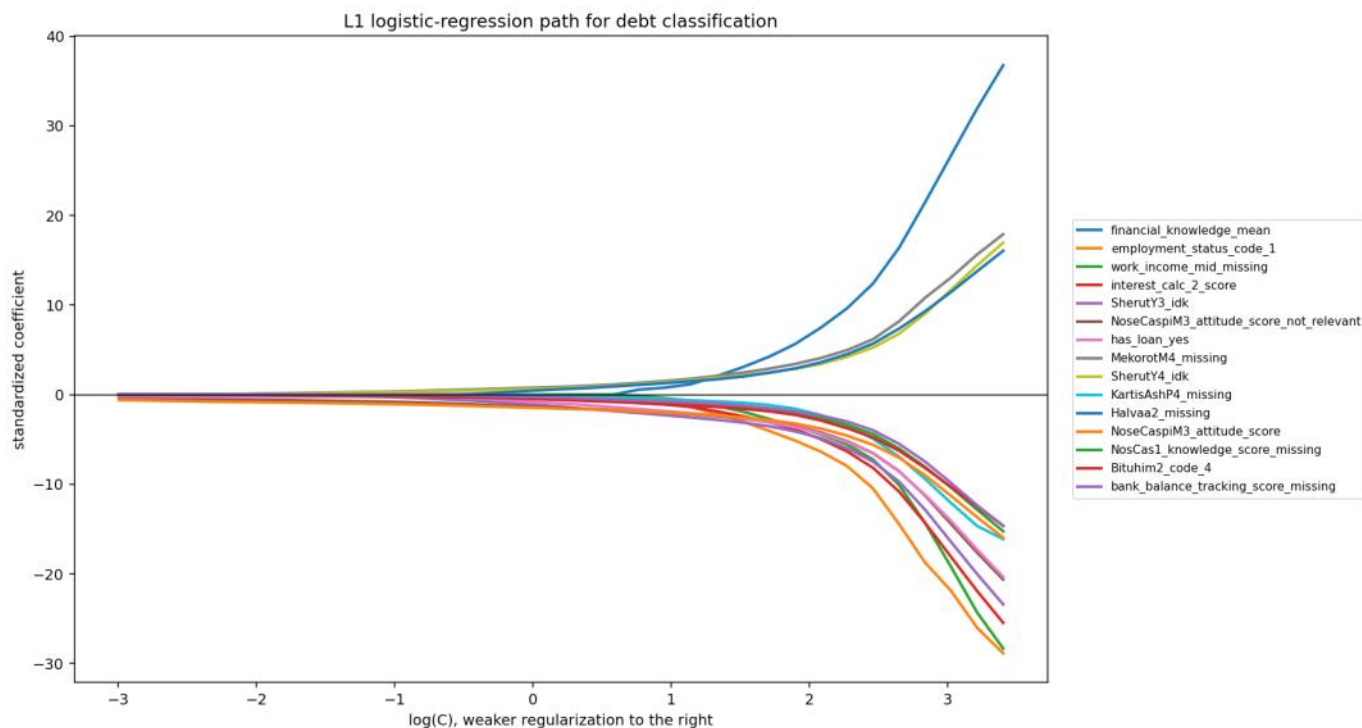
2. Lasso Path for Classification

For the Lasso path, we used an L1-regularized logistic regression model.

Since this is a classification task, and not regression, the standard regression Lasso path is not the correct model for this question. Before fitting the path, features were standardized, and constant features were dropped since their standard deviation is zero.

Main facts:

- constant features dropped: 23
- features used in the Lasso path: 421
- number of regularization values: 35
- active coefficients at strongest regularization: 0
- active coefficients at weakest regularization: 177



Top 5 selected features details:

- financial_knowledge_mean: A combined proxy for financial knowledge, averaging several knowledge-related survey scores; its importance suggests that general financial understanding is predictive of debt status.
- employment_status_code_1: An indicator for one employment-status category; its selection suggests that employment situation contains useful information for predicting whether a person has debt.
- work_income_mid_missing: An indicator that the respondent's personal work-income value is missing or unknown; this suggests that non-response or lack of reported work income is itself predictive.
- interest_calc_2_score: A score based on the respondent's answer to an interest-calculation question; it reflects financial numeracy and appears predictive of debt status.
- SherutY3_idk: An indicator that the respondent answered "I don't know" for service question 3; this suggests that uncertainty or lack of familiarity with that financial service is associated with the debt label.

From the Lasso path suggests that the strongest predictive signals come

from financial knowledge variables, employment/income variables, loan-related variables, financial-attitude variables, and several missingness or 'IDK' indicators. However, these variables should be interpreted as predictive features, not as causal explanations for debt.

3. PCA

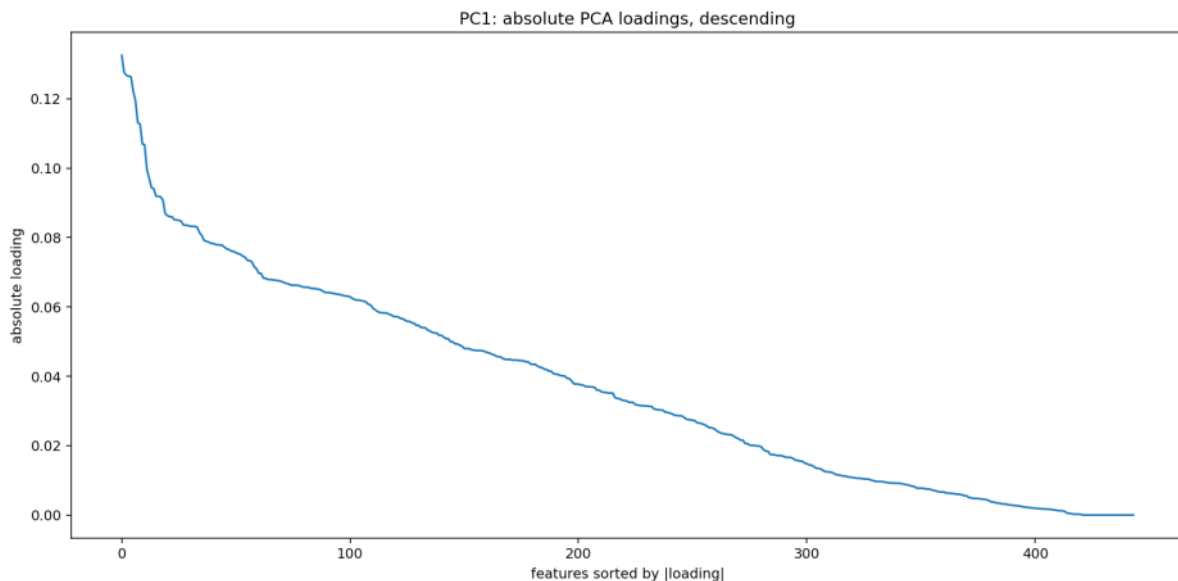
The PCA part was used to understand which groups of features explain large variation in the feature matrix X. PCA was applied after standardizing the features.

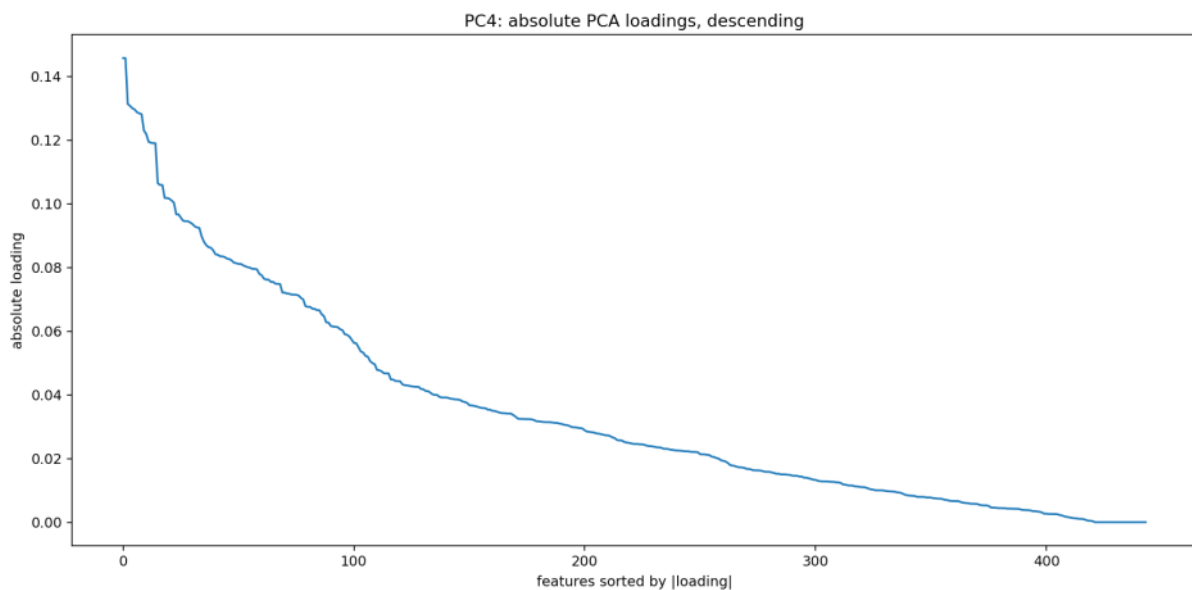
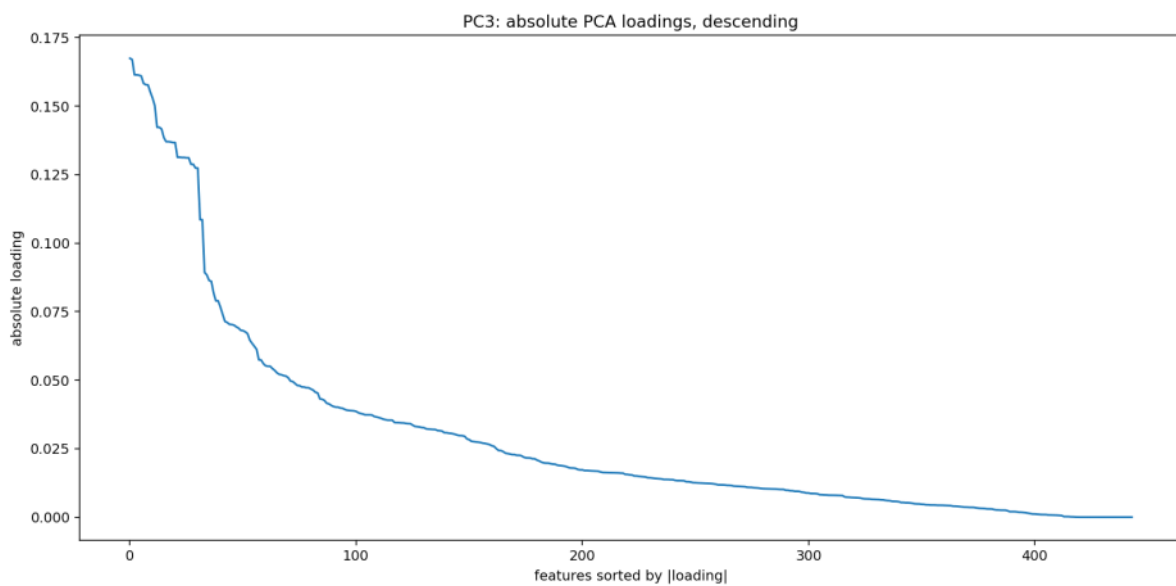
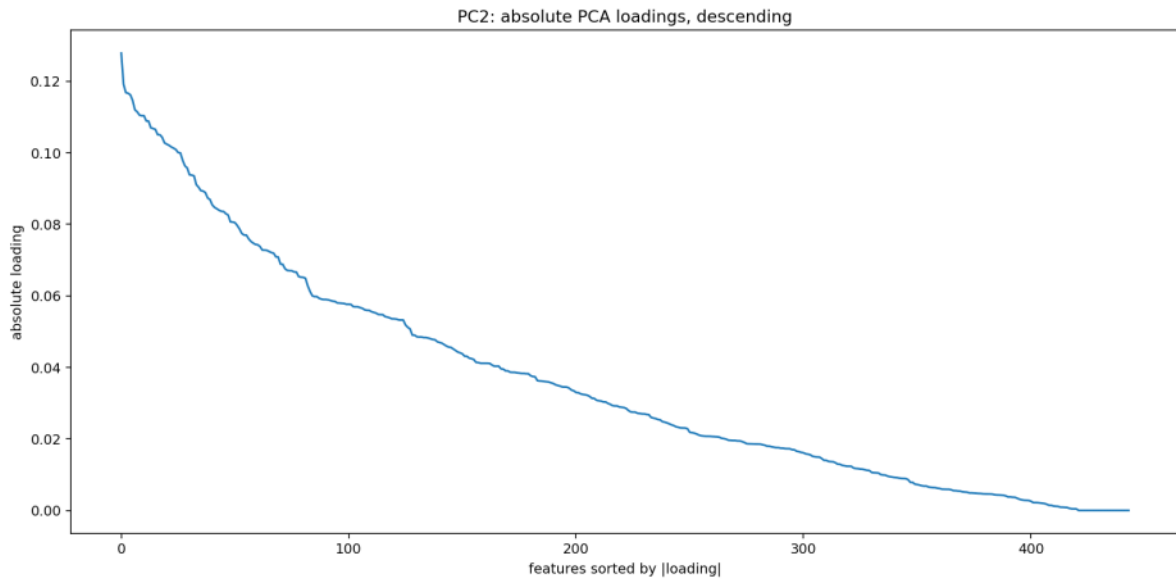
a. Explained variance

Component	Explained variance	Cumulative explained variance
PC1	7.10%	7.10%
PC2	5.50%	12.60%
PC3	3.37%	15.97%
PC4	3.22%	19.19%

The first four principal components explain about 19.2% of the total standardized feature variance. This means that the variation in the data is not concentrated in only one or two directions. This is reasonable because the data contains many one-hot encoded variables, score variables, and missingness indicators.

b. PCA loading plots





Each plot shows the absolute PCA loadings sorted in descending order. A large absolute loading means that the corresponding feature contributes strongly to that principal component.

Summary of dominant structures:

Component	Main loading pattern
-----------	----------------------

PC1	credit-card question missingness / response pattern
PC2	investment/savings response pattern and IDK indicators
PC3	loan-related variables and loan-question missingness
PC4	IDK aggregates and loan-related response patterns

c. PR-AUC relevance to debt

The positive class is debt, and its prevalence is:

$$230 / 1153 = 0.1995$$

Therefore, a random ranking would have PR-AUC approximately equal to 0.20.

Component	PR-AUC	Lift over baseline
PC1	0.232	+0.033
PC2	0.229	+0.030
PC3	0.282	+0.082
PC4	0.334	+0.135

PC1 and PC2 are only slightly above the random baseline, so they are not very relevant to the debt label. PC3 is more relevant, and PC4 is the most relevant among the first four components, with PR-AUC 0.334. Still, this is only a moderate result. Since PCA is unsupervised, the directions with highest variance are not necessarily the directions that best predict debt.

We checked the top PCA-loading features and did not find direct Hovot debt-detail branch variables among them. However, many high-loading variables are still response-pattern or missingness indicators, so the interpretation should remain cautious.

4. Supervised Prediction Models

The data was split into approximately 2/3 training and 1/3 testing.

Set	Total	No debt	Debt
Train	768	615	153
Test	385	308	77

We trained three model families:

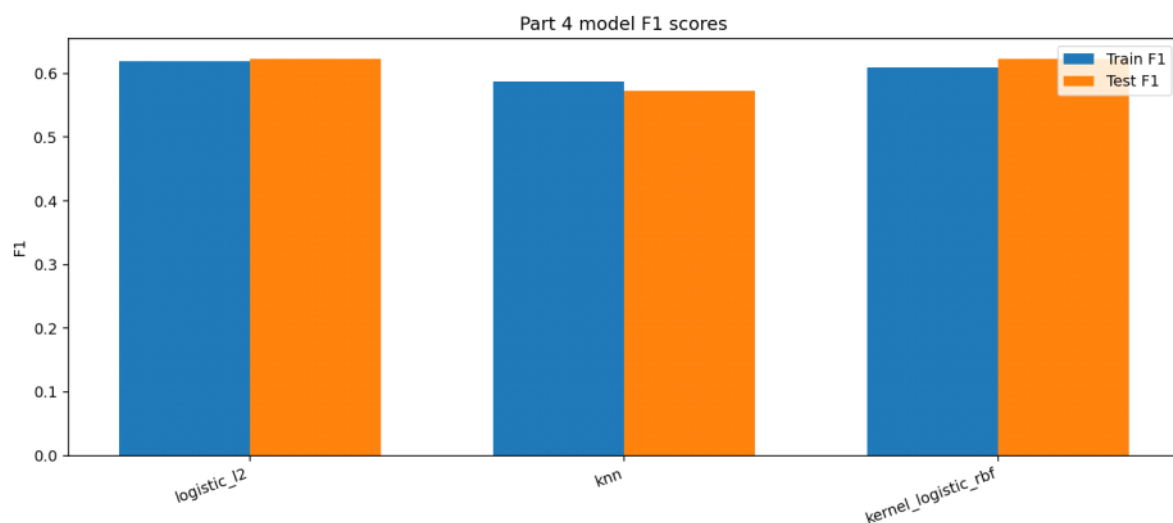
1. L2 logistic regression
2. K-nearest neighbors
3. RBF-kernel logistic regression

Hyperparameters and classification thresholds were selected using 5-fold stratified cross-validation on the training set, using positive-class F1 score. The KNN model used Euclidean distance. Because the feature space is sparse and high-dimensional after one-hot encoding, KNN is expected to be more sensitive to irrelevant/noisy features than logistic regression.

We also compared different numbers of Lasso-selected features. In the held-out test set, the top-15 feature set performed best for the three model families. Larger feature sets often increased training F1 but did not improve test F1, suggesting that adding more selected features may add noise or overfitting.

Final results:

Model	Feature set	CV F1	Train F1	Test F1
Logistic L2	Lasso top 15	0.5983	0.6188	0.6234
KNN, Euclidean	Lasso top 15	0.5760	0.5879	0.5731
RBF kernel logistic	Lasso top 15	0.5993	0.6101	0.6225



Logistic regression and RBF-kernel logistic regression achieved almost the same held-out test F1, around 0.62. KNN was weaker, with test F1 around 0.57. This fits the expectation that distance-based methods may struggle in a sparse engineered feature space (the curse of dimensionality we learned in class). The final train-test gaps are not very large, suggesting that the compact Lasso-selected feature set helped reduce overfitting.

5. Learner Diversity and Majority Vote

For the three final learners, we built an error matrix E on the training set. Each row represents one training observation, and each column represents one model. The entry is 1 if the model made a wrong prediction and 0 otherwise.

After centering and normalizing the columns, we computed: $\tilde{E}^T \tilde{E}$

The resulting matrix was:

Model	Logistic	KNN	Kernel logistic
Logistic	1.0000	0.6798	0.9226
KNN	0.6798	1.0000	0.7083
Kernel logistic	0.9226	0.7083	1.0000

The diagonal entries are 1 because each model has perfect similarity with

itself. The off-diagonal values measure similarity between the error patterns of different models. The values are all positive and fairly high, especially between logistic regression and kernel logistic regression. This means that the models often make mistakes on the same observations, so the diversity between them is limited.

Majority vote results:

Model	Train F1	Test F1
Logistic L2	0.6188	0.6234
KNN	0.5879	0.5731
RBF kernel logistic	0.6101	0.6225
Majority vote	0.6192	0.6184

Majority voting improved over KNN, but it did not improve over the best individual model. The best individual test F1 was logistic regression with 0.6234, while majority vote achieved 0.6184. This is consistent with the error-correlation matrix: since the learners make similar mistakes, simple voting does not add much new information.

A better combination could be weighted voting or stacking. In stacking, a second-level model could be trained on the predicted probabilities of the three base models, preferably using out-of-fold predictions. This would allow the meta-learner to give more weight to the stronger models instead of treating all three votes equally.

Summary

Overall, the supervised-learning results show that debt prediction is possible but not extremely strong. The best models reached test F1 around 0.62. Lasso and PCA both showed that response patterns, missingness indicators, financial knowledge, attitude variables, and loan/card-related variables contain useful predictive information. However, many of these variables reflect survey structure, so they should be interpreted carefully. The majority-vote ensemble did not improve over the best individual model, probably because the learners had positively correlated errors.

Unsupervised Learning

Thursday, 2 July 2026 8:25

1. Following the spirit of the supervised part, We started with mapping the fields in the two tables.

The Crimes Dataset Contains per-city crime records. Each record contains:

- o FictiveIdNumber -some non-unique identifier for the crime or possibly the suspect
- o the yearly quarter when the crime occurred (categorized Q1-Q4)
- o the YeshuvKod that serves as the join key with the cities dataset
- o Police organizational hierarchy - district, zone and station. Assumingly they handled the case.
- o Crime category with 15 types
- o Crime subtype with 158 subtypes
- o Municipality name and sub-municipal name - seems empty for some records

The Cities dataset contains all official settlements in Israel, of all municipal types (city/Moshav/Kibutz/etc). Each entry contains:

- o City name and ID
- o District
- o Municipal type
- o Religious type
- o Population
- o Ethnicity
- o Police station code
- o Geo coordinates including height

2. Before building the graph and performing the spectral analysis, we thought to pre-process the data to avoid unwanted statistical artifacts. The preprocessing steps we took were:

- a. Removing crime rows with no YeshukKod
- b. Removing crime rows with "error" (שגיאת הונה) value, "other" 1200 code value, and the 1300 code values which don't seem crime-related.
- c. Joined the two tables
- d. Removing city rows with missing population or coordinates.
- e. Remove tiny city entries with less then 2000 population count
- f. Parsing the Geographical values as ITM coordinates

We also chose to work with the StatisticTypeKod rather then the StatisticGroupKod, which seemed finer and therefore more interesting to analyze (even though it created more outliers which effected the preprocessing).

3. For the similarity computation, we wanted to both normalize population sizes and also address the crime profile per city rather then its intensity. To do so, we normalized each city by its population, and afterwards normalized that vector 1 so we'd see the crime mix.

$$v_i = \left[\frac{C_{i,1}}{Pop_i} \quad \dots \quad \frac{C_{i,n}}{Pop_i} \right], \text{ where } C_{i,j} \text{ is the crime rate of type } j \text{ in city } i, Pop_i \text{ is the total population in city } i, \text{ and } C_i = (C_{i,1} \dots C_{i,n})$$

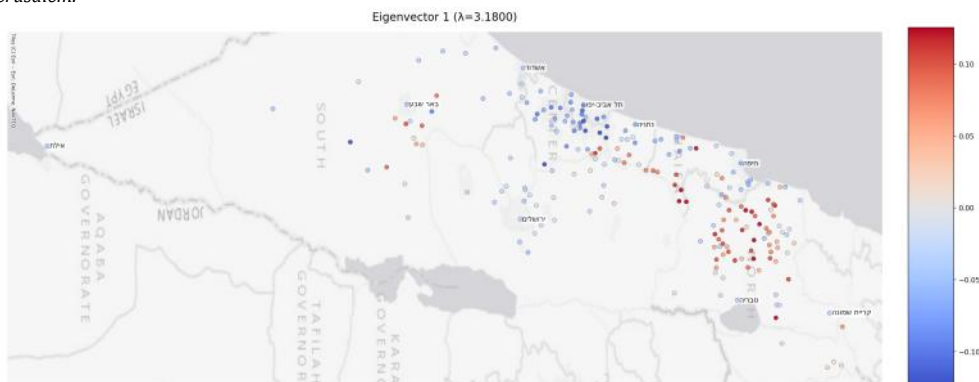
After calculating the embedding, we used the standard RBF kernel for distance:

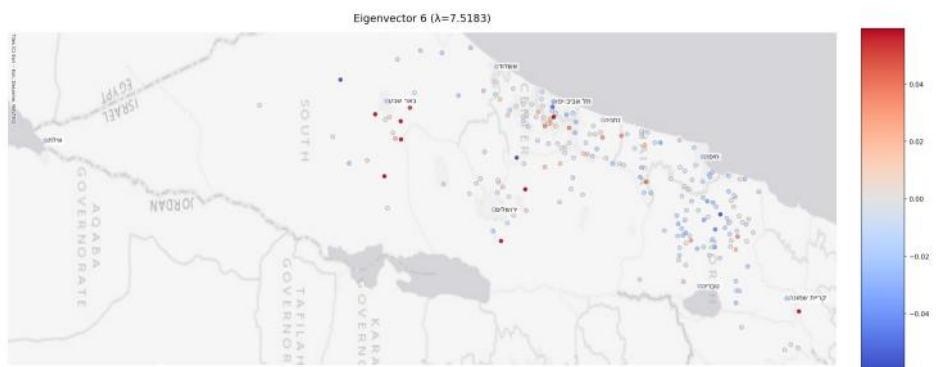
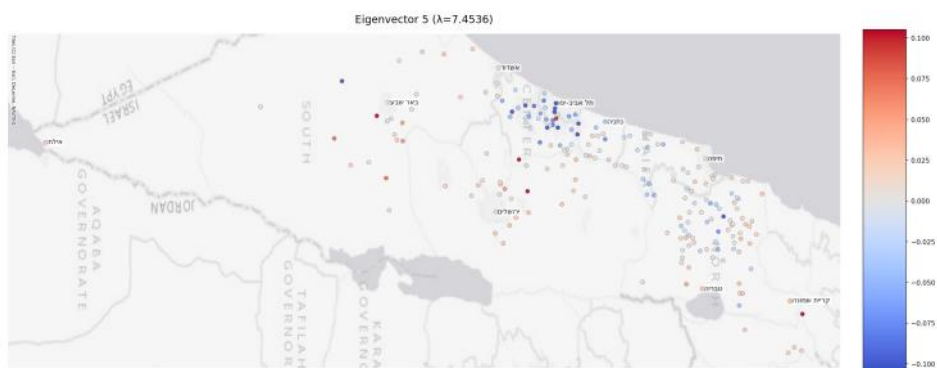
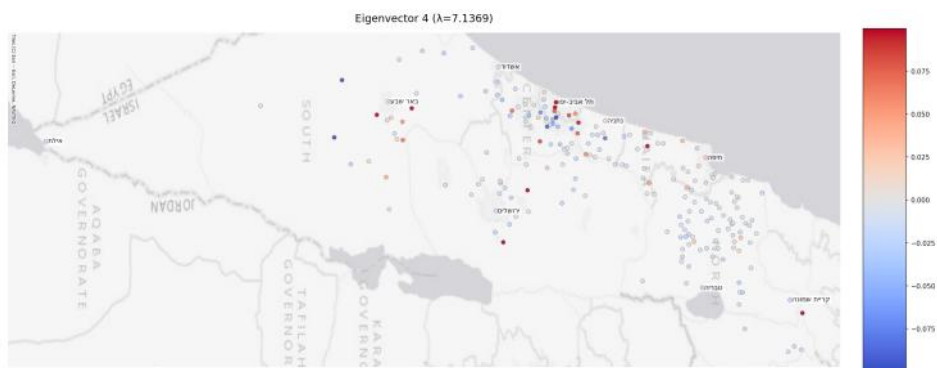
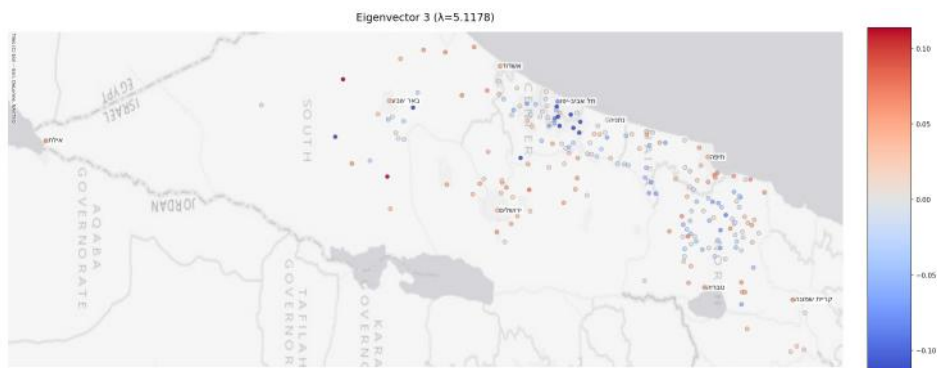
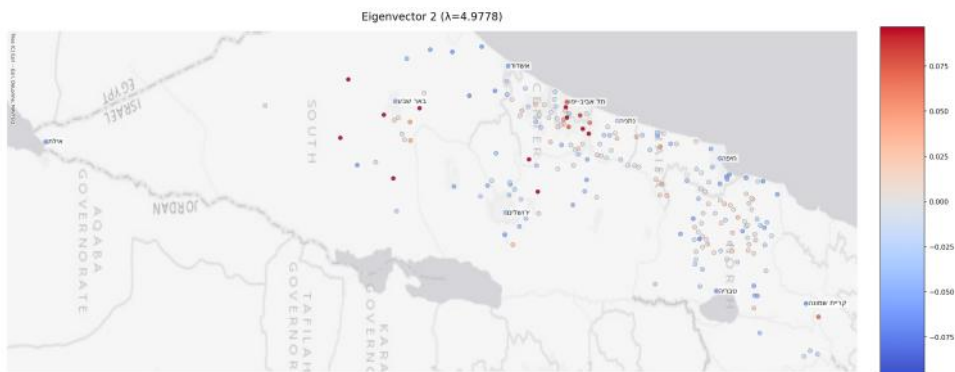
$$w_{ij} = \exp\left(-\frac{|v_i - v_j|^2}{\sigma_i \sigma_j}\right), \text{ with } \sigma_i \text{ calculated per-city by calculating the distanc to its 7th nearest neighbor, as we saw in "A Tutorial on$$

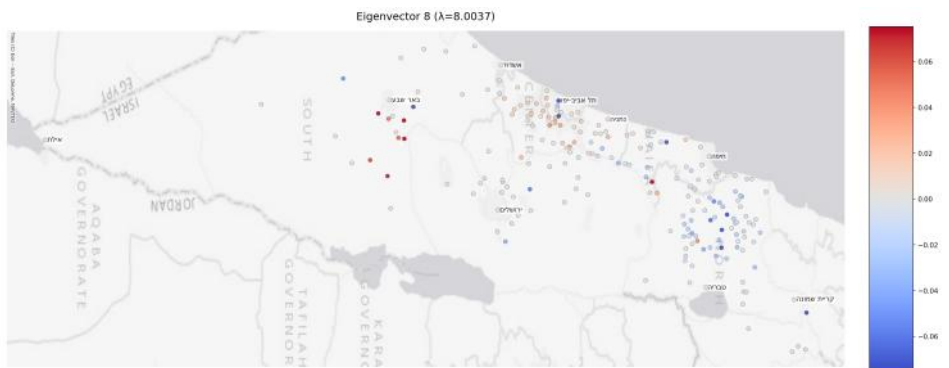
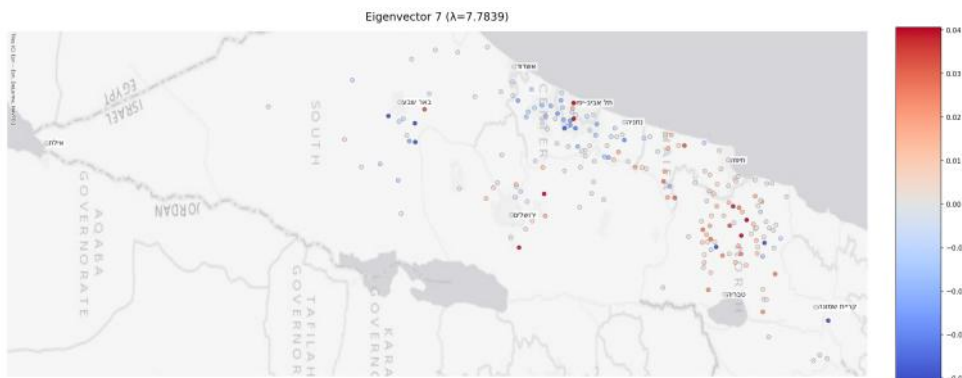
Spectral Clustering" from Moodle (that we learned in class).

4. (and 6.) Looking at the eigenvectors, We don't see a clear single city that dominates each vector, but we defintely do see regions in the country. More on this liater. Here are some of the Leading eigenvectors from the Laplacian of the graph:

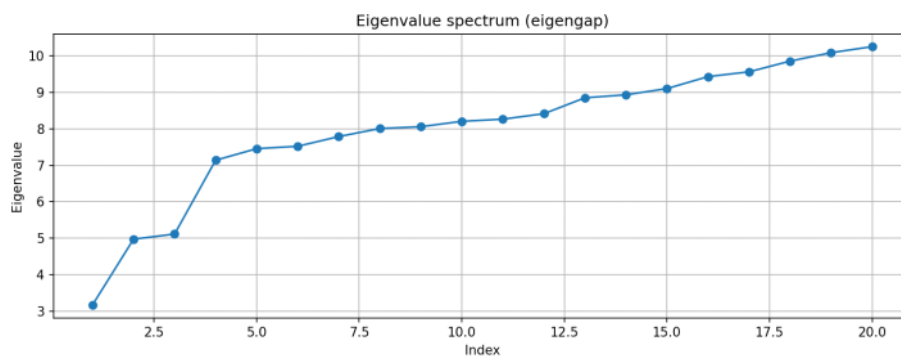
EV1's hottest region correspond to Central Galilee area. EV2 is centered at Tel Aviv and the Sharon area. EV3 is more mixed and shows Jerusalem area and also parts of Haifa's close area. EV4-EV6 emphasize different parts of Gush Dan and Hashfela, as well as areas near jerusalem.



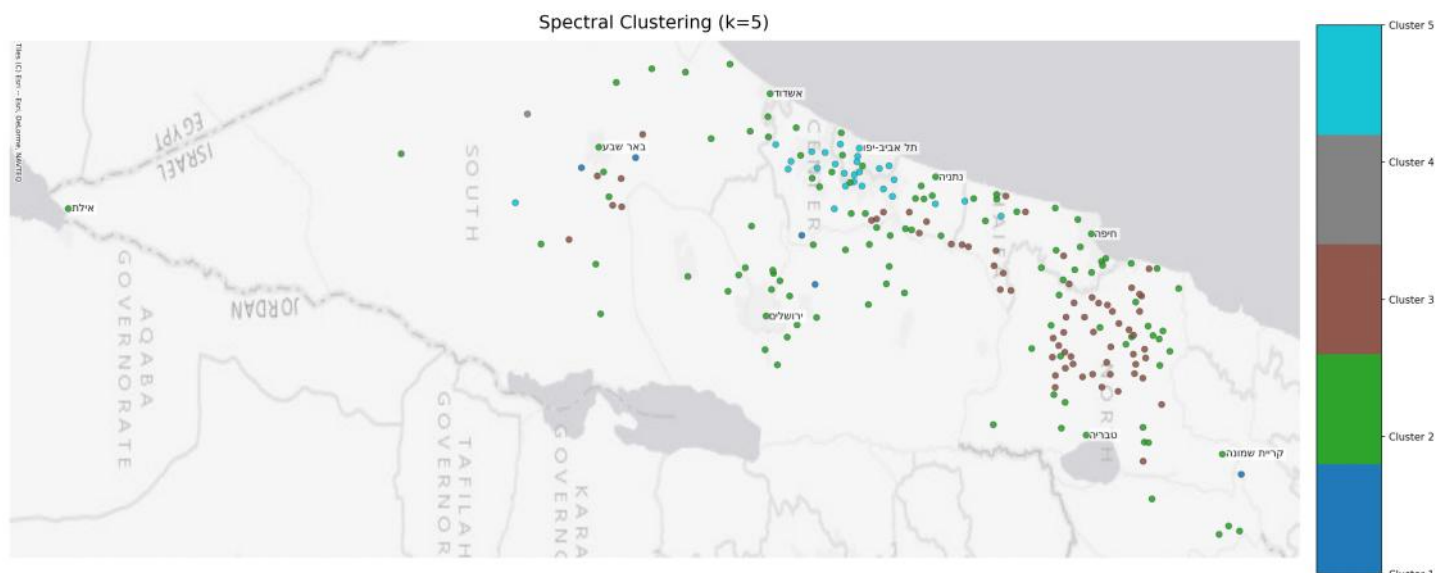




5. To select the correct K for Spectral clustering, we first looked at the eigengap graph:



We see the values still rising up to a plateau at around 5, so as we learned, the "jumps" between plateaus represent between-cluster variations, and plateaus are in-cluster variation. Based on the graph, we chose $k=5$.



Somewhat similarly to the eigenvectors themselves, the clusters here correspond to different geographical areas in Israel - Cluster 3 is mostly in the Galilee villages eastern to Beer Sheva, Cluster 2 is spread throughout most of the country, Cluster 4 is Beer Hadaj only, and cluster 5 maps nicely to Gush Dan and Hasharon.

Even more interesting - looking at the ethnic representation in each cluster - we see cluster 3 with an astonishing 98% Arab cities. Here's the full table:

Cluster ID	Jewish Pop	Arab Pop	Mixed Pop	Number of cities in cluster
1	60%	40%	0	5
2	80%	14%	6%	108
3	2%	98%	0%	65
4	0	100%	0	1
5	96%	0%	4%	27

Since we normalized the crime intensities of each cities, we compared the crime profiles of the cities regardless of their size. We essentially managed to (partially) recreate the ethnicity column, which corresponds strongly with the big clusters 2 (Jewish) and 3 (Arab).

We also searched for other correlations buy using spearman correlation (for more robust results), and found religion, municipal status and population to be strongest correlators. They all represent ethnical and socio-economic differences, so this strengths our clustering.

Multiple Hypothesis Testing

Thursday, 2 July 2026 13:24

After performing the unsupervised analysis, we wondered whether there were specific types of crimes that started increasing or decreasing in occurrence relative to a few years back. We expected that some types of crime would become more prominent following the war and rise of organized crimes in the Arab society.

For the data we used, we downloaded the crimes data set for the years 2022 and 2025 to be able to compare. To make this interesting, we looked at all 136 crime subtypes across the years. For each of these crime types, we computed the per-capita crime rate in each city, in each year, and compared the two city-level values from these two years.

The preprocessing steps we did were similar to the ones in the unsupervised part - both are done to reduce noise caused by bad values or tiny villages that could be outliers. We didn't need to normalize the data in any way, as opposed to the unsupervised part.

Our Null hypothesis:

Crime type j had similar per-capita rate in 2022 and 2025.

Our Alternative hypothesis:

Crime type j per-capita rate is different in 2022 and 2025.

$H_0: \mu_{j,22} = \mu_{j,25}, H_1: \mu_{j,22} \neq \mu_{j,25}$, where $\mu_{j,i}$ is per-capita rate of crime type j across all cities in year i .

Our Test: we went with a Welch 2-sample t test. Welch was chosen over student t -test, in order to allow different variances for the 2 populations.

In addition, since each observation is city-rate per-type per-year, and we have 206 cities, then we can probably be safe when using the CLT as justification for the normal approximation used in the welch t -test.

$$T_j = \frac{\bar{x}_{j,22} - \bar{x}_{j,25}}{\sqrt{\frac{s_{j,22}^2}{n_{22}} + \frac{s_{j,25}^2}{n_{25}}}}, \text{ with } \bar{x}_{j,i} \text{ being the sample mean for crime } j \text{ in year } i, s_{j,i} \text{ being}$$

the sample SD for crime j in year i , and n_i is the number of observations in year i .

We calculated the p -value from the standard calculation of 2-sided p value for two-sample t -test :

$$p_j = 2P(t_v \geq |T_i|)$$

Multiple Hypothesis Testing:

Since we performed this test over 136 crime types, we'd expect to get $136 * 0.05 \approx 6.8$ false positives for $\alpha = 0.05$. To address that without being too conservative, we used Benjamini-Hochberg procedure as we learned in class:

1. After calculating the p-values, we sorted them ascending.
2. We calculated the BH threshold for each rank:

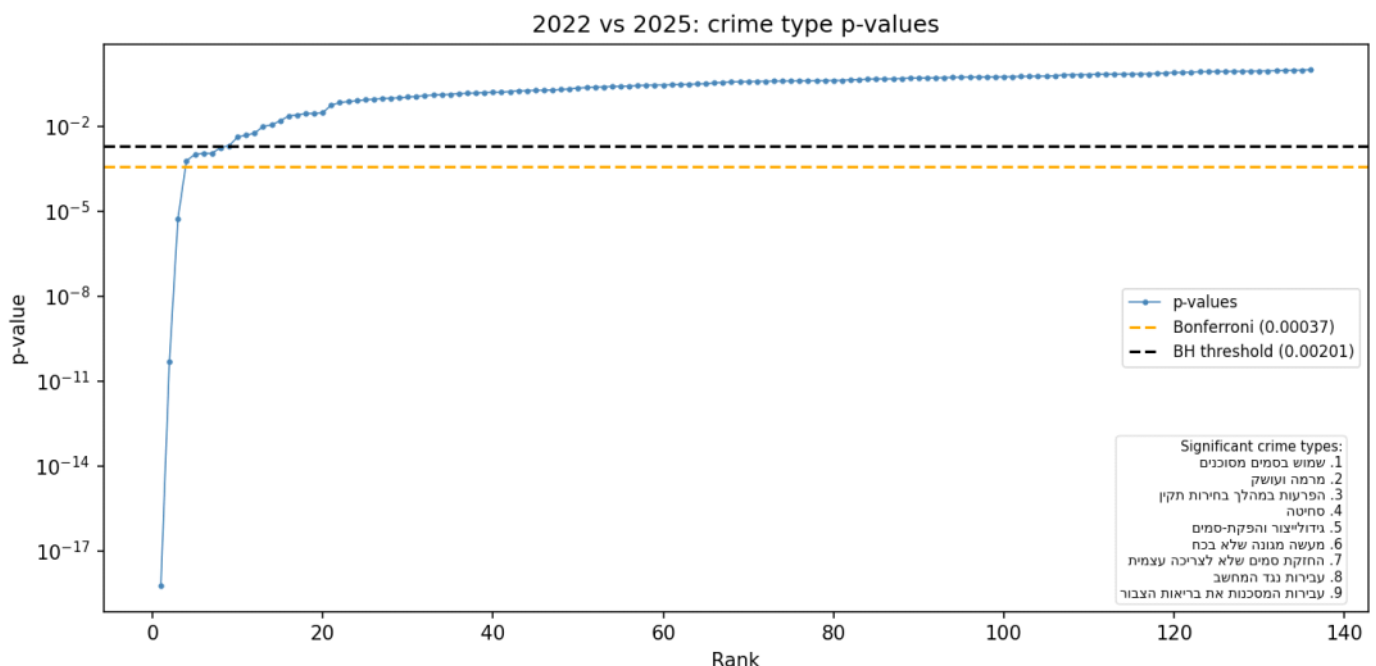
$$\frac{k}{m}\alpha, k \in [m]$$

3. Found $k_{\max} = \max k \text{ s.t. } p_k \leq \frac{k}{m}\alpha$
4. Rejected all null hypothesis for $k_{\max}$ and below it.

We preferred BH over Bonferroni to get for "free" more significant results, while keeping FDR below the wanted threshold so we won't "cheat" by p-hacking.

Results:

Using BH, we found 9 crime types that changed significantly in proportion in 2025, relative to 2022. To plot this we took this graph from our exercise 2 code:



From a sociological point of view, some of these results make sense - 2022 was an election year so we'd expect more election interference crimes, and if we recall correctly several laws about de-criminalization of cannabis were introduced around 2022.

From a statistical point of view, we can see that using Bonferroni would have been more conservative, and in which case we'd only have found 3 significant crime types.

Overall, we did manage to find some crime types that changed their prevalence between these 3 years.

Transformers

Wednesday, 8 April 2026 17:06

Part A: Architecture Design

We designed a transformer-style model for graph-level molecular classification. Each input sample is a molecular graph, where nodes represent atoms and edges represent chemical bonds. The goal is to predict one binary label for the whole molecule: whether the molecule inhibits HIV replication.

Tokenization

We use the **graph nodes as tokens**. In the molecular graph, each node corresponds to one atom, so each atom is treated as one token in the transformer. For example, if a molecule has 25 atoms, then the transformer receives 25 node tokens. Unlike text, these tokens do not have a natural order, so the model must use graph structure and graph positional information rather than ordinary sequence position.

Token embedding

The node features in ogbg-molhiv are categorical atom-level features. Therefore, we do not treat the feature values as continuous numerical values.

Instead, each categorical node feature has its own embedding table. For each atom, we embed each of its categorical features and sum the embeddings to obtain the initial token representation.

If atom i has categorical features: $x_i = (x_{i1}, x_{i2}, \dots, x_{i9})$ then its initial embedding is: $h_i = \text{Emb}_1(x_{i1}) + \text{Emb}_2(x_{i2}) + \dots + \text{Emb}_9(x_{i9})$

This gives every atom a vector in the model dimension d_{model} . This is similar to word embeddings in NLP, but here the embedding represents atom properties rather than words.

Graphs with different numbers of nodes

Different molecules have different numbers of atoms. To handle this, we use **padding and masks** inside each batch.

For a batch of molecules, all graphs are padded to the number of nodes of the largest graph in that batch. A node mask is created to distinguish real atoms from padded positions.

The mask is used in two places:

1. **Attention:** padded nodes are not allowed to participate as real tokens.
2. **Pooling:** padded nodes are ignored when computing the graph representation.

So the model can process a batch as a tensor, while still correctly handling molecules of different sizes.

Use of graph structure

A standard transformer allows every token to attend to every other token, but in a molecular graph we also know which atoms are chemically bonded. We use this information through an **attention bias**.

The model uses the adjacency matrix and bond features to modify the attention

scores. For every pair of atoms, if there is a real bond between them, the attention score receives an additional learned bias. The edge features are also embedded and converted into attention-head-specific bias values.

So the attention score has the form:

$$\text{attention score}(i, j) = \text{query-key similarity} + \text{bond/adjacency bias}(i, j)$$

This keeps the transformer global, because every atom can still attend to every other atom, but it tells the model which atom pairs are directly connected by bonds.

Positional encoding

Graphs do not have a natural left-to-right order like text. Therefore, ordinary sinusoidal sequence positional encoding is not suitable.

We used **Laplacian positional encoding**. For each molecule, we compute the normalized graph Laplacian from its adjacency matrix:

$$L = I - D^{-\frac{1}{2}}AD^{-\frac{1}{2}}$$

Then we compute the first nontrivial eigenvectors of this Laplacian. These eigenvectors give each node a structural coordinate inside the graph.

The Laplacian positional encoding is fixed for each graph, because it is computed from the graph structure. However, before adding it to the node embeddings, we pass it through a trainable linear layer, so the model can learn how to use this structural information.

So the input to the transformer is:

node token embedding + projected Laplacian positional encoding

Attention blocks

The model uses standard transformer-style blocks, adapted to graphs.

Each block contains:

1. multi-head self-attention over the node tokens;
2. adjacency/bond attention bias added to the attention scores;
3. masking of padded nodes;
4. residual connection;
5. layer normalization;
6. feed-forward MLP;
7. another residual connection and layer normalization.

The attention layers update each atom representation by allowing it to aggregate information from other atoms in the molecule, while giving special treatment to bonded atom pairs through the edge/adjacency bias.

Dropout is used during training inside the transformer blocks, but it is disabled during validation and test evaluation.

Graph-level prediction head

After the transformer blocks, the model has one final vector representation for each atom: $h_1, h_2, \dots, h_n$

But the label is a graph-level label, not a node-level label. Therefore, we need to combine the node representations into one molecule representation.

The best model used **mean+max pooling**.

Mean pooling captures the average/global information across all atoms:

$$h_{\text{mean}} = \frac{1}{n} \sum_{i=1}^n h_i$$

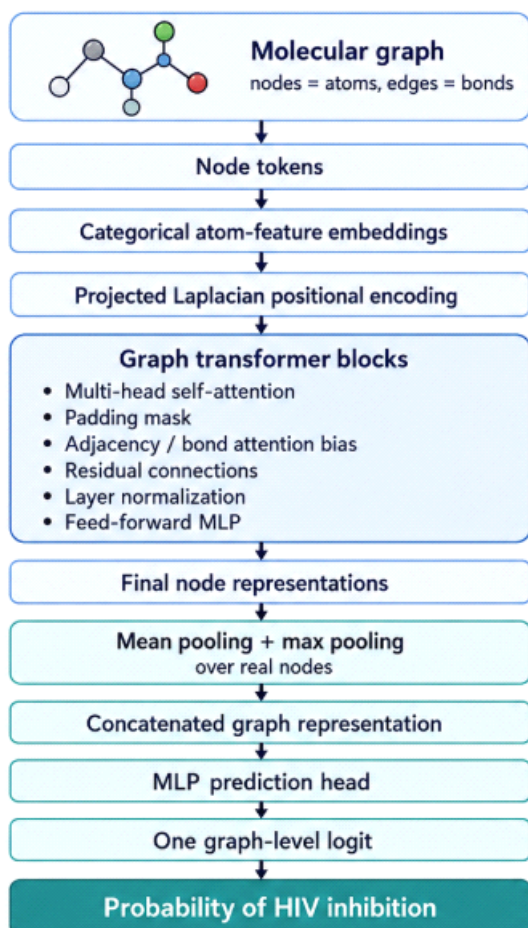
Max pooling captures the strongest local signal in any atom representation:

$$h_{\text{max}} = \max\{h_1, h_2, \dots, h_n\}$$

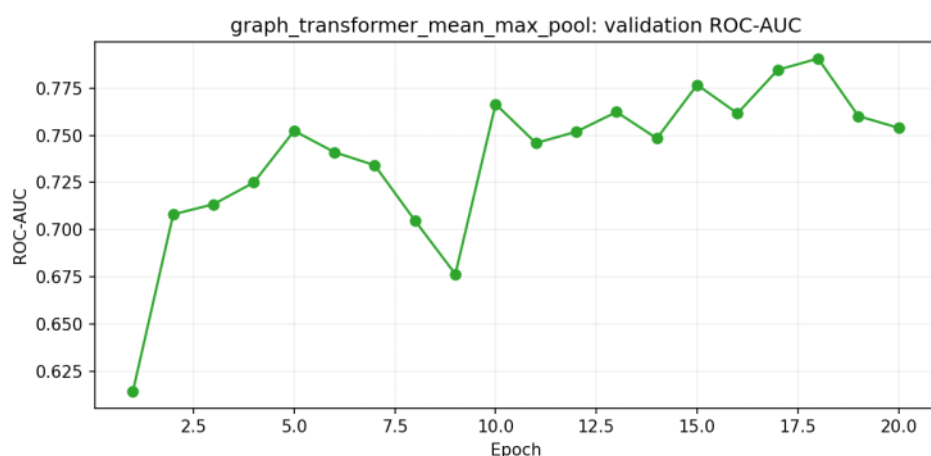
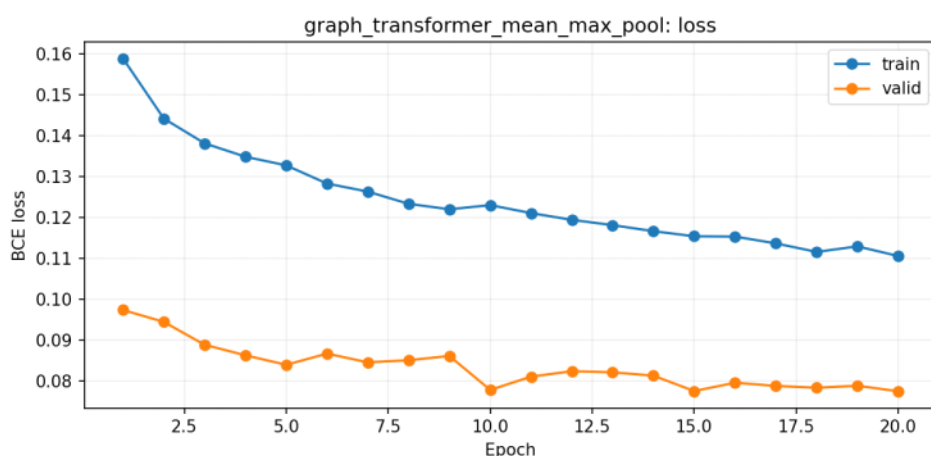
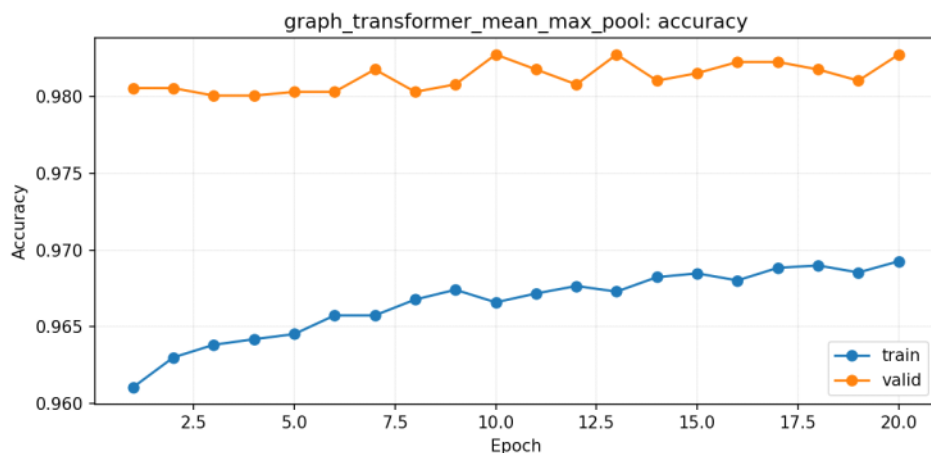
Then we concatenate them: $h_{\text{graph}} = [h_{\text{mean}}, h_{\text{max}}]$

This graph representation is passed into an MLP classifier that outputs one logit.

The logit is used with "BCEWithLogitsLoss" during training, and sigmoid is applied when converting the logit into a probability.



Part B: Results



The best architecture was the graph transformer with mean+max pooling. The model was trained for 20 epochs, and the best epoch was selected according to validation ROC-AUC. The validation ROC-AUC increased during training and reached its maximum value of 0.7907 at epoch 18. The corresponding test ROC-AUC was 0.7566. The BCE training loss decreased steadily, showing that the optimization was working. The validation loss was lower than the training loss, probably because validation was evaluated with dropout disabled, while training loss was measured during training mode. Accuracy was high for both training and validation, but because the dataset is imbalanced, ROC-AUC was used as the main metric. Overall, the model learned useful molecular representations, with reasonable generalization to the test set.

LLM Usage

Thursday, 2 July 2026 13:13

We marked all cells that contain LLM-generated code with "Auxiliary" comments, so it's easy to distinguish. This is how we used LLMs to assist us in the process:

1. Supervised learning part:

- a. We used ChatGPT/Codex to help with technical and auxiliary coding tasks, mainly organizing the preprocessing pipeline, reading the Hebrew-encoded survey files, exporting the processed CSV files, and creating audit files for free-text answers.
- b. We used it to help generate utility code for saving outputs, creating plots, arranging the code into notebook cells, and keeping the notebook structure consistent with the rest of the project.
- c. We used it to help implement mechanical parts of the workflow such as one-hot encoding, missing-value flags, output tables, and plot formatting

2. Unsupervised part:

- a. We used Claude to load the data in a similar way to the given ipynb file, and split it to the relevant data-frames we used in the project
- b. We used it also to create all the plots and make the visual edits so they look good
- c. We used Claude to make the table in `correlate_features` look visually correct.

3. Multiple Hypothesis part:

- a. We used Claude to load the data, and copy the preprocessing steps we did for the unsupervised part
- b. We used it to copy the graph from Exercise 2 that shows the p-values sorted by rank

In both the Unsupervised and Multiple hypothesis parts - we originally wrote the code in `.py` files, and used Claude to rearrange them as code cells in the `ipynb` file.

4. Transformers:

- a. We used ChatGPT/Codex to generate the implementation code for the graph-transformer experiment according to our selected design.
- b. We used it to load the ogbg-molhiv dataset, implement padded graph batches, cache Laplacian positional encodings, save plots/results/model weights, and organize the code in the notebook.
- c. We used it to debug Colab-related technical issues, including Google Drive saving and PyTorch/PyG loading compatibility.